```
const txt = "Let's break stuff";

const testScope = function () {
    console.log(txt);
}

testScope();
```

The code above can be executed via Docker using the following bash command:

```
$ docker run -it --rm --name my-running-script -v "$PWD":/usr/src/app \
  -w /usr/src/app node:19-bullseye node js_scope.js
```

The above assumes the code is stored in the file `js_scope.js` and the output is simply "Let's break stuff" without any warnings. In JavaScript, local scope has access to parent scope. Not so with PHP.

There is in fact a way for `testScope()` to access `$txt` from global scope, by using the keyword `global`, or the superglobal variable `$GLOBALS`. Here are two ways to access global scope from a local scope in PHP:

```
$txt = 'Greetings from global scope' . PHP_EOL;
testScope1();
testScope2();

function testScope1(): void
{
    global $txt;
    print $txt;
}

function testScope2(): void
{
    print $GLOBALS['txt'];
}

Output:
Greetings from global scope
Greetings from global scope
```